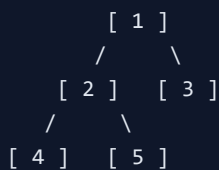


Binary Tree Traversals (Inorder, Preorder, Postorder, Level Order) — Data Structures

Definition: Tree Traversal is the systematic process of visiting (reading, processing, or printing) every node in a tree data structure exactly once.

1. Detailed Technical Explanation



1. Depth-First Traversals (DFS):

1. Preorder Traversal (Root -> Left -> Right):

- Visit Root, Traverse Left Subtree, Traverse Right Subtree.

- Example Output: **1 -> 2 -> 4 -> 5 -> 3**

2. Inorder Traversal (Left -> Root -> Right):

- Traverse Left Subtree, Visit Root, Traverse Right Subtree.

- Example Output: **4 -> 2 -> 5 -> 1 -> 3**

- Key Property: Inorder traversal of a **Binary Search Tree (BST)** always produces elements in **sorted ascending order**!

3. Postorder Traversal (Left -> Right -> Root):

- Traverse Left Subtree, Traverse Right Subtree, Visit Root.

- Example Output: **4 -> 5 -> 2 -> 3 -> 1**

- Key Property: Used in Expression Tree evaluation and deleting tree nodes from bottom-up.

2. Breadth-First Traversal (BFS / Level Order):

- Visits nodes level-by-level from top to bottom, and left to right at each level using a **FIFO Queue**.

- Example Output: **1 -> 2 -> 3 -> 4 -> 5**

2. Complete Executable C Program for Traversals

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int val) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}

// 1. Inorder Traversal: Left -> Root -> Right
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// 2. Preorder Traversal: Root -> Left -> Right
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// 3. Postorder Traversal: Left -> Right -> Root
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    // Construct sample binary tree
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("Preorder : "); preorder(root); printf("\n");
    printf("Inorder : "); inorder(root); printf("\n");
    printf("Postorder : "); postorder(root); printf("\n");
}
```

```
    return 0;  
}
```

3. Reconstructing Unique Binary Tree from Traversals

- A unique binary tree can be constructed if and only if **INORDER** is given along with either **PREORDER** or **POSTORDER**.
 - Inorder splits left and right subtrees; Preorder/Postorder identifies the root node.
-

4. Must-Write Points for Exams

- Inorder traversal of BST gives non-decreasing sorted order.
 - Time Complexity of all traversals: **$O(N)$** since each node is visited once.
 - Auxiliary Space: **$O(H)$** where **H** is tree height (for recursion call stack).
-

5. Quick Recall Flow

Preorder (V-L-R) | Inorder (L-V-R) | Postorder (L-R-V) | Level Order (Queue BFS)